Análise SOLID - Ponto Fácil

Análise do código atual, sem refatorar ainda

O objetivo desta análise é olhar o que já está pronto no Ponto Fácil e identificar onde o código já está bem separado e onde pode melhorar usando SOLID. O projeto ainda está sendo construído por partes, então alguns componentes do C3 representam o que queremos ter no sistema completo e ainda não foram implementados.

1. Análise dos 5 princípios SOLID

SRP - Responsabilidade Única

Situação atual: O projeto já separa Route, Validator, Controller, Service e Model.

Violação encontrada: EmployeeService e UserService ainda fazem mais de uma coisa. Além da regra de negócio, também acessam banco e mexem com arquivos/fotos.

Alteração proposta: Separar acesso ao banco em Repository e arquivos em um serviço próprio. O Service ficaria mais focado na regra de negócio.

OCP - Aberto/Fechado

Situação atual: A divisão em módulos facilita adicionar novas partes no projeto.

Violação encontrada: Se a forma de salvar fotos ou acessar o banco mudar, hoje precisamos alterar diretamente os Services.

Alteração proposta: Criar abstrações para armazenamento e persistência. Assim poderíamos trocar a implementação sem mexer tanto na regra principal.

LSP - Substituição de Liskov

Situação atual: Ainda não usamos várias implementações de uma mesma interface ou classe base.

Violação encontrada: Não encontrei uma violação concreta hoje.

Alteração proposta: Quando criarmos Repository, Storage ou outras abstrações, as implementações precisam poder ser trocadas sem quebrar o Service.

ISP - Segregação de Interfaces

Situação atual: Ainda não existem interfaces formais de Repository ou Storage.

Violação encontrada: Não existe hoje uma interface gigante obrigando classes a implementar métodos que não usam.

Alteração proposta: Quando criarmos interfaces, manter contratos pequenos e específicos, como EmployeeRepository, UserRepository e FileStorage.

DIP - Inversão de Dependência

Situação atual: Controllers chamam Services, o que já está organizado.

Violação encontrada: Os Services dependem direto de implementações concretas, como Employee/User do Sequelize, fs, path e bcrypt.

Alteração proposta: Fazer os Services dependerem de abstrações e configurar as implementações no ponto de entrada da aplicação.

2. Respostas às questões da atividade

1. Cada componente do C3 tem módulo correspondente? Parcialmente. User, Employee, Routes, Controllers, Validators e Models já existem. Time Record, Validation e Facial Service ainda são partes planejadas.

2. A regra de negócio está isolada da infraestrutura? Não completamente. Os Services ainda acessam diretamente Sequelize/Models, arquivos e bcrypt.

3. O C4 coincide com as dependências do código? O fluxo coincide: Route -> Middleware/Validator -> Controller -> Service -> Model. Porém ainda usamos require() direto e não injeção de dependência.

4. Cada classe/módulo tem uma única responsabilidade? Parcialmente. Routes, Validators e Controllers estão bem separados, mas os Services acumulam mais responsabilidades.

5. Foram evitadas God Classes? Sim, por enquanto. Não existe uma classe central fazendo tudo. O cuidado é não deixar os Services crescerem demais.

6. Validação e persistência estão separadas? Parcialmente. Validação está nos Validators e upload no Multer. Persistência ainda é chamada diretamente pelos Services.

7. Dá para adicionar novos comportamentos sem alterar o código atual? Parcialmente. Novos módulos podem ser criados, mas trocar banco ou armazenamento de fotos ainda exige mexer nos Services.

8. Mudanças de requisito evitam mexer na regra central? Nem sempre. Algumas mudanças ainda obrigam alterar EmployeeService ou UserService.

9. LSP está sendo atendido? Ainda não se aplica de forma forte, porque não temos várias implementações do mesmo contrato.

10. As interfaces são pequenas e específicas? Ainda não temos interfaces formais para avaliar isso.

11. Existem métodos vazios por obrigação de interface? Não encontrei esse problema.

12. Services dependem de interfaces? Não. Esse é um dos principais pontos de DIP a melhorar.

13. As dependências são montadas no ponto de entrada? Ainda não. O app.js monta Express e rotas, mas os Services importam as dependências por conta própria.

3. Principais violações encontradas no código

employeeService.js

Esse é o melhor exemplo para estudar SRP, OCP e DIP, porque no mesmo arquivo temos regra de funcionário, banco e manipulação de fotos.

1) Arquivos e fotos dentro do Service: o arquivo importa fs e path, monta o caminho da pasta e apaga fotos antigas.

```
const fs = require('fs')
const path = require('path')

function deleteEmployeePicture(filename) {
  ...
  if (fs.existsSync(filePath)) {
    fs.unlinkSync(filePath)
  }
}
```

2) Regra de CPF junto com banco: o Service procura o CPF diretamente com Employee.findOne() e também cria o funcionário com Employee.create().

```
const cpfExists = await Employee.findOne({
  where: { cpf },
})

if (cpfExists) {
  const error = new Error('Este CPF já está cadastrado.')
  ...
}

const employee = await Employee.create({
  fullName,
  cpf,
  ...
})
```

3) Edição mistura várias responsabilidades: na mesma função ele busca o funcionário, confere CPF, atualiza o banco e depois apaga a foto antiga.

```
const employee = await getEmployeeById(id)
const cpfExists = await Employee.findOne(...)
await employee.update(...)

if (profilePicture && oldPicture !== profilePicture) {
  deleteEmployeePicture(oldPicture)
}
```

O problema não é o código não funcionar. Ele funciona. A questão do SOLID é que esse Service pode precisar ser alterado por vários motivos diferentes. Por isso a proposta é separar banco e arquivos da regra de negócio.

userService.js

No UserService acontece algo parecido. Ele conhece o Model User, Sequelize, bcrypt e também fs/path.

```
const User = require('./userModel')
const bcrypt = require('bcryptjs')
const { Op } = require('sequelize')
const fs = require('fs')
const path = require('path')
```

No cadastro ele consulta o banco, cria o hash da senha e salva o usuário. Na atualização de perfil também pode apagar foto antiga. Então também temos mistura entre regra de usuário e detalhes de infraestrutura.

Partes que já estão bem separadas

Nem tudo precisa ser alterado. Algumas partes já seguem bem a ideia da arquitetura em camadas:

- EmployeeController recebe a requisição e chama o Service.
- EmployeeValidator fica responsável pela validação dos dados.
- EmployeeMulter cuida do recebimento e limite das imagens.
- Routes definem os endpoints e a ordem dos middlewares.

4. Vestígios do CivicWatch

O Ponto Fácil foi criado a partir de uma cópia do CivicWatch, porque os dois projetos estão sendo feitos em paralelo e o CivicWatch serve como base. Por isso ainda existem alguns restos no código.

```
function globalSearch(query) {
  return { politicians: [], users: [] };
}
```

O termo politicians é do CivicWatch e não faz parte do Ponto Fácil. Esse ponto será removido depois. Existem também alguns comentários antigos, mas eles não mudam o funcionamento do sistema.

5. C3 planejado x implementação atual

O C3 representa o sistema completo que queremos chegar. Por isso ele mostra alguns componentes que ainda não foram implementados. Na análise SOLID estou considerando somente o código que já existe.

- Já existe: Routes, Middlewares/Validators, Controllers, User Service, Employee Service e Models.
- Ainda será feito: Time Record Service, Validation Service, Facial Service e integração com a API facial.